

5. 프롬프트 평가 자동화 스크립트 작성

- **LLM 기반 프롬프트 자동 평가 스크립트 작성:** GPT-4o-mini 모델을 중심으로, 필요에 따라 GPT-4o를 활용하여 프롬프트와 그 응답에 대한 품질 평가를 자동화합니다. LLM을 평가자로 활용하여 사람이 일일이 채점하지 않아도 **정확성, 일관성** 등의 기준으로 응답 품질을 판정합니다.
- **평가 결과 분석 및 비교:** 자동 평가된 결과를 Pandas 데이터프레임으로 정리하고, 막대 그래프 등의 **시각화 도구**로 분석 및 비교합니다. 이를 통해 모델별 성능 차이나 프롬프트 유형별 응답 품질 경향을 파악하고자 합니다.

1. 평가 자동화 개요

평가 대상: 프롬프트와 그에 대한 LLM의 응답 쌍이 평가 대상입니다. 수강생들이 생성한 여러 프롬프트-응답 결과물을 데이터셋으로 삼아 평가 스크립트가 각 쌍의 품질을 채점하도록 합니다.

평가 항목: 이번 실습에서는 단순한 **정답/오답**과 많이 쓰이는 **지표**를 사용해 봅니다. 일반적으로 LLM 응답 품질 평가는 정확성, 유창성, 일관성, 관련성 등 다양한 지표로 수행됩니다.

LLM 기반 평가 프로세스: 사람이 아닌 **LLM에게 평가를 위임**하는 과정입니다. 이를 위해 **평가 지침을 담은 시스템 프롬프트**를 구성하고, **평가할 프롬프트-응답 쌍**을 입력으로 제공합니다. LLM은 지침에 따라 각 평가 항목별 점수를 산정하고, 그 결과를 JSON과 같은 구조화된 형식으로 반환합니다. 이러한 **LLM-as-a-Judge** 접근법은 평가를 자동화하여 효율을 높이며, 사람이 개입하지 않아도 **다양한 측면(정확성, 일관성 등)을 동시에 평가**할 수 있다는 장점이 있습니다. 특히 **출력 형식을 JSON으로 지정**하면 이후 프로그래밍적으로 파싱하여 분석하기 수월합니다.

2. Colab 기반 평가 스크립트 실습

이 절에서는 Google Colab에서 Python을 활용하여 **정답/오답** 자동 평가를 수행하는 코드를 단계별로 구현합니다. **OpenAI API를 통해 GPT 모델을 호출**하고, 앞서 정의한 **평가 기준**에 따라 응답을 채점하도록 **프롬프트를 구성**합니다.

##실습 5.1. Google Colab 실습

- **OpenAI API 설정:** Colab 환경에서 OpenAI API 키를 불러오고 클라이언트를 초기화합니다. 예를 들어, 아래 코드처럼 `google.colab` 모듈의 `userdata`를 통해 **환경 변수에 저장된 API 키**를 읽어옵니다:

```
!sudo apt-get install -y fonts-nanum
# 폰트 설치 후 Colab 메뉴에서 런타임/세션 다시 시작을 선택합니다.

!pip install -U openai

from google.colab import userdata
import os
import openai

# Colab에 저장한 Secret에서 API 키 가져와 환경 변수에 등록
os.environ["OPENAI_API_KEY"] = userdata.get('OPENAI_API_KEY')
```

최신 OpenAI 파이썬 라이브러리가 설치되었다고 가정하며, 클라이언트 인스턴스를 생성합니다:

```
client = openai.OpenAI()
```

이후 API 호출을 통해 얻은 `response` 객체에서 모델의 응답 텍스트는 `response.choices[0].message.content` 와 같이 접근할 수 있습니다.

- **평가 프롬프트 구성:** 이제 **LLM에게 평가 작업을 지시하는 프롬프트**를 만듭니다. 시스템 메시지는 평가자 역할로서 지켜야 할 기준과 각 점수의 의미를 상세히 설명합니다. 예컨대 시스템 프롬프트에 다음과 같은 내용을 포함할 수 있습니다: "당신은 답변 심사위원입니다. 사용자에게 제공된 **프롬프트와 그 응답**을 읽고, **정확성, 일관성, 포맷 적절성** 각 항목에 대해 1~5점으로 평가하세요. 평가 기준: 5점은 매우 우수, 1점은 매우 미흡. **JSON 형식으로만 답변하세요.**" 이러한 지침 뒤에 실제 평가 대상인 프롬프트와 응답을 함께 제공하면, 모델이 이를 읽고 채점하도록 유도합니다. 여기서는 조금 쉽게 접근할 수 있도록 **정답/오답**으로 처리하도록 하겠습니다.

```
def make_evaluation_prompt(prompt, answer):
    return f"""
        프롬프트: {prompt}
        응답: {answer}

        위 프롬프트에 대한 응답을 정답/오답으로 답변해주세요.
        정답/오답만 답변합니다.
        """
```

- **LLM 호출 및 응답 확인:** 준비된 평가 프롬프트를 OpenAI API로 GPT-4o 또는 GPT-4o-mini에 전송하여 **정답** 혹은 **오답** 여부를 확인합니다.

```
prompts_and_answers = [
    ("프랑스의 수도는 어디인가요?", "파리"),
    ("전세계에서 가장 높은 산은 어디인가요?", "에베레스트"),
    ("1 더하기 3은 얼마인가요?", "5")
]

for prompt, answer in prompts_and_answers:
    eval_prompt = make_evaluation_prompt(prompt, answer)

    response = client.chat.completions.create(
        model = "gpt-4o-mini",
        messages = [
            {"role": "user", "content": eval_prompt}
        ]
    )

    content = response.choices[0].message.content
    print(f"Prompt: {prompt}, Answer: {answer}, Evaluation: {content}")

    if content == '오답':
        score = {"correct": False}
    else:
        score = {"correct": True}
    # You can do something with the score here, e.g., store it in a list
    print(f"Score: {score}")
```

- **반복 평가 루프 구성:** 하나의 프롬프트-응답 쌍에 대한 평가가 정상적으로 이루어졌다면, 이를 여러 데이터에 반복 적용하도록 코드를 작성합니다. 예를 들어 여러 프롬프트에 대한 여러 모델의 응답이 리스트 형태로 주어졌다면, `for` 루프를 통해 순차적으로 평가를 수행할 수 있습니다:

```
results = []
for prompt, answer in prompts_and_answers:
    # 프롬프트에 대한 답변이 맞았는지 평가를 요청
    eval_prompt = make_evaluation_prompt(prompt, answer) # 프롬프트
```

와 응답으로 평가 프롬프트 생성

```
response = client.chat.completions.create(
    model = "gpt-4o-mini",
    messages = [
        {"role": "user", "content": eval_prompt}
    ]
)

content = response.choices[0].message.content
print(content)

if content == '오답':
    score = {"correct": False}
else:
    score = {"correct": True}
score["prompt"] = prompt # 원본 프롬프트나 식별자 저장
score["answer"] = answer
score["model"] = "gpt-4o-mini"
results.append(score)
```

위 코드에서는 `prompts_and_answers` 를 순회하며 각 응답을 평가하고, `results` 리스트에 평가 결과를 누적합니다. 이때 결과마다 어떤 모델의 응답인지 식별할 수 있도록 `score` 딕셔너리에 `"model"` 정보를 추가했습니다. 만약 **GPT-4o와 GPT-4o-mini 두 모델을 비교**한다면, 각 모델에 대해 같은 프롬프트 세트를 응답 생성 후 평가하여, 모델 구분 필드를 넣어가며 `results` 에 추가하면 됩니다.

- **Pandas DataFrame으로 수집 및 저장:** 루프 완료 후 `results` 에는 모든 평가 결과가 딕셔너리들의 리스트로 담깁니다. 이를 Pandas의 DataFrame으로 변환하면 표 형태로 다루기가 용이합니다:

```
import pandas as pd
df = pd.DataFrame(results)
df # 데이터프레임 출력
```

이렇게 하면 DataFrame의 컬럼으로 `correct, prompt, answer, model` 의 정보가 들어갑니다. 필요에 따라 `df.to_csv('evaluation_results.csv', index=False)` 와 같이 CSV 파일로 저장하여 추후 사용하거나 공유할 수도 있습니다.

3. 평가 결과 시각화 실습

이 섹션에서는 자동 평가된 결과를 다양한 **시각화 기법으로 분석**합니다. Pandas와 Matplotlib 등을 활용하여 점수를 요약하고 비교하는 그래프를 그려볼 것입니다. 코드를 통해 직접 시각화를 수행하고, 결과 이미지를 확인하면서 통찰을 얻습니다.

- **지표별 평균 점수 비교:** 먼저 전체 데이터에 대해 각 평가 항목(정확성, 일관성, 포맷)의 **평균 점수**를 계산하고 막대 그래프로 나타낼 수 있습니다. 예를 들어 `df[["정확성", "일관성", "포맷"]].mean()` 을 구하면 각 컬럼별 평균을 얻을 수 있고, 이를 `plt.bar` 등을 이용해 시각화합니다. 이 막대그래프를 통해 모델의 전반적인 응답 품질에서 어떤 측면이 가장 우수하거나 미흡한지 한눈에 파악할 수 있습니다. 특정 지표의 평균 점수가 유난히 낮다면 해당 부분에서 모델 성능 개선이 필요함을 시사합니다.
- **모델별 평가 점수 비교:** 두 가지 모델(GPT-4o-mini vs GPT-4o)을 사용했다면, **모델별로 점수를 분리하여 비교**하는 것도 중요합니다. Pandas에서는 `df.groupby("model").mean()` 으로 모델별 평균 점수를 계산할 수 있습니다. 이를 이용해 **그룹 막대 그래프**를 그리면 동일한 지표에 대해 어떤 모델이 더 좋은 성능을 보였는지 쉽게 비교할 수 있습니다. 아래 그래프는 예시로 각 지표에 대해 두 모델의 평균 점수를 시각화한 것입니다.

예시: GPT-4o-mini(노랑)와 GPT-4o(주황) 모델의 평가 점수 비교 그래프. 그래프는 각 평가 지표(정확성, 일관성, 포맷)에 대한 두 모델의 평균 점수를 나타낸다. 이 예시에서 GPT-4o 모델이 전반적으로 GPT-4o-mini보다 높은 점수를 받아 우수한 성능을 보이는 것을 확인할 수 있다. 이러한 시각화는 모델별 성능 차이를 한눈에 파악하고 분석하는 데 도움을 준다.

- **프롬프트 유형별 응답 품질 비교:** 추가로, 프롬프트를 몇 가지 **카테고리**로 분류해두었다면 (예: 질의응답형, 창의적 글쓰기형, 코드 관련 요청 등), **유형별로 응답 품질을 분석**해볼 수도 있습니다. DataFrame에 프롬프트 유형 정보를 포함시켰다면 `df.groupby("prompt_type").mean()` 으로 유형별 평균 점수를 구하고 그래프로 표현할 수 있습니다. 이를 통해 특정 유형의 프롬프트에 대해 모델이 약점을 보이는지 또는 강점을 보이는지 파악 가능하며, 필요시 프롬프트 설계를 개선하는 근거가 됩니다.
- **기타 시각화 기법:** 점수 분포를 살펴보기 위해 히스토그램을 그리거나, 두 점수 간 상관관계를 보기 위해 **산점도**를 그려보는 등 추가적인 분석도 가능합니다. 예를 들어 **정확성 점수와 일관성 점수 사이에 양의 상관관계가 있는지 확인**하면, 모델이 사실상 오류를 낼 때 맥락 일관성도 무너지는 경향이 있는지 해석할 수 있습니다. 이러한 다각도의 시각화로 평가 결과를 입체적으로 분석합니다.

4. 결과 해석 및 개선 방향 도출

시각화된 결과를 토대로 **모델의 장단점 및 프롬프트 설계 개선점**을 논의합니다. 우선, **지표별 점수 편차**를 해석합니다. 어떤 평가 항목은 평균이 낮거나 모델 간 차이가 크게 나타났다면, 그 이유를 생각해봐야 합니다. 예를 들어:

- 정확성 점수가 낮았다면 모델이 **사실과 다른 정보를 생성**하는 경향(환각)이 문제일 수 있습니다. 이런 경우 **지식 기반 강화**(예: 정보 검색 활용)나 **프롬프트에 사실 검증 단계 추가** 등의 개선을 고려합니다.
- 일관성 점수 편차가 크다면 **역할 지시 또는 톤 통제**에 있어 불안정함이 있는 것입니다. 시스템 메시지를 통해 역할을 더욱 분명히 정의하거나, 대화 기록을 충분히 제공하여 맥락을 유지하는 등의 조치를 생각해볼 수 있습니다.
- 포맷 적절성에서 점수가 낮다면 모델이 출력 형식 요구를 제대로 따르지 않은 경우입니다. 이 경우 **프롬프트에서 형식 관련 지시**를 더 강조하거나, 모델이 자주 실수하는 형식을 찾아 별도로 후처리하는 방안을 고려합니다.

모델별 비교 해석: GPT-4o-mini와 GPT-4o 간에 성능 차이가 관찰되었다면, 각 모델의 특성으로 인한 것인지 분석합니다. 가령 GPT-4o가 정확성 면에서 일관되게 높은 점수를 받았다면, 더 많은 파라미터와 훈련 데이터로 사실적 응답을 생성하는 능력이 우수하기 때문일 것입니다. 반대로 GPT-4o-mini에서 특정 유형 프롬프트에 더 좋은 성능을 보였다면 경량 모델 특유의 **빠른 응답**이나 **튜닝된 도메인 지식** 영향일 수 있습니다. 이러한 분석을 통해 **모델 선택 또는 조합 전략** (예: 중요한 상황에서는 GPT-4o 사용, 일반적 응답은 GPT-4o-mini로 속도 향상) 등을 도출합니다.

프롬프트 튜닝 및 평가 프로세스 개선: 평가 과정에서 드러난 문제점은 곧 프롬프트 설계 개선으로 이어질 수 있습니다. 예를 들어 일관성 평가에서 반복적으로 낮은 점수가 나온다면, 프롬프트에 **시스템 역할 지시**를 강화하거나 사용자에게 받은 컨텍스트를 명시적으로 나열하는 등 **프롬프트 엔지니어링**으로 해결을 시도합니다. 또한 LLM 평가자 자체의 편향이나 한계를 보완하기 위해, **평가 프롬프트**를 다듬는 것도 중요합니다. 필요하다면 평가 LLM에게 평가 근거를 함께 내놓도록 유도하여(체점의 설명 추가) 결과를 신뢰하지만, 이번 세션에서는 단순 점수 산출에 집중합니다. 마지막으로, 인간 평가와 LLM 평가 결과를 일부 샘플에서 비교해보는 것도 권장됩니다 – LLM 평가가 사람이 보기에 그릴듯한지 검증함으로써 자동 평가의 **신뢰성**을 판단할 수 있습니다.

5. 참고 자료

실습을 돕기 위해 **전체 평가 및 시각화 스크립트**가 Colab 노트북으로 제공됩니다. 노트북에는 앞서 설명한 단계들이 모두 구현되어 있으며, 수강생이 직접 코드를 실행하면서 결과를 확인할 수 있도록 구성되어 있습니다. 또한 **평가 기준을 정의한 프롬프트 예시**(시스템 메시지와 입력 형식)와, 샘플 데이터에 대한 **시각화 출력 예시**가 포함되어 있습니다. 이를 참고하여 수강생들은 자신만의 프롬프트 및 모델 응답에 대한 평가를 시도해보고, 필요에 따라 지표나 평가 프롬프트를 수정해보면서 LLM 평가의 유용성과 한계를 체험해 보시기 바랍니다.

각 단계를 진행하면서 궁금한 점이나 추가로 시도해보고 싶은 아이디어가 있다면 자유롭게 토론합니다. 이번 세션을 통해 자동화된 평가 기법으로 **프롬프트 엔지니어링**을 개선하고, 시각적 분석으로 **LLM 응답 품질**을 심층적으로 이해하는 방법을 마스터하는 것이 목표입니다.

참고: LLM을 활용한 평가 기법(LLM-as-a-Judge)은 최근 많은 연구와 실용 사례에서 등장하는 주제입니다. 예를 들어 OpenAI의 GPT-4 같은 강력한 모델을 평가자로 활용하면 비용은 들지만 일관된 기준 적용이 가능하며, JSON과 같은 구조화 출력으로 손쉽게 다량의 결과를 분석할 수 있습니다. 수강생들은 제공된 자료 외에도 관련 최신 사례 연구를 살펴보고, 자신만의 프로젝트에 적용할 방안을 생각해보면 좋겠습니다.